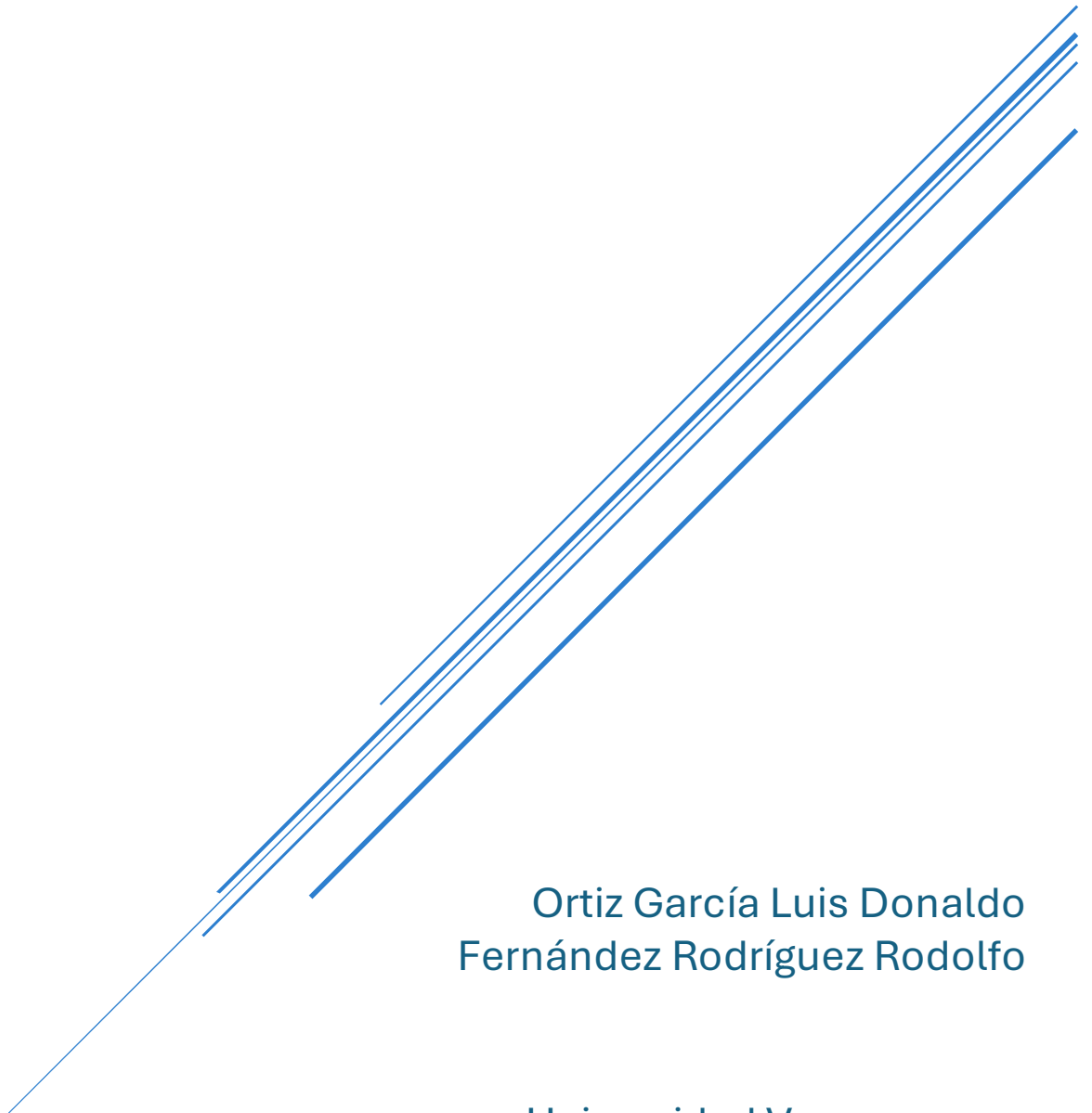


LAYLA WORLDBUILDING

Especificación funcional



Ortiz García Luis Donaldo
Fernández Rodríguez Rodolfo

Universidad Veracruzana
Desarrollo de sistemas en red

Contenido

INTRODUCCIÓN	2
PROPÓSITO	2
ALCANCE	2
DEFINICIONES, ACRÓNIMOS Y ABREVIACIONES.....	3
REFERENCIAS.....	4
CONTEXTO	4
CLASES DE USUARIO	5
CASOS DE USO	6
DEFINICIÓN	6
CLASIFICACIÓN	7
DESCRIPCIONES DE CASOS DE USO	8
<i>Módulo de Acceso y Seguridad</i>	<i>8</i>
<i>Módulo de Gestión y Seguridad</i>	<i>12</i>
<i>Módulo de Worldbuilding.....</i>	<i>17</i>
<i>Módulo de Voz y Colaboración.....</i>	<i>19</i>
<i>Módulo de Lectura Pública.....</i>	<i>22</i>
PROTOTIPO IU.....	26
REQUISITOS FUNCIONALES	28
MÓDULO DE GESTIÓN DE USUARIOS Y SEGURIDAD (BACKEND .NET + SQL SERVER)	28
MÓDULO DE ESCRITURA Y PROYECTOS (BACKEND NODE.JS + MONGODB)	28
MÓDULO DE WORLDBUILDING Y RELACIONES (BACKEND NODE.JS + NEO4J).....	28
MÓDULO DE COLABORACIÓN Y VOZ (ANDROID + WEBSOCKETS)	29
MÓDULO DE LECTURA Y ACCESO PÚBLICO (WEB RAZOR PAGES)	29
MÓDULO DE ADMINISTRACIÓN Y REPORTES (WEB RAZOR PAGES)	29
REQUISITOS NO FUNCIONALES.....	30
OPERABILIDAD Y DESPLIEGUE	30
RESTRICCIONES.....	31
RIESGOS (¿QUÉ PODRÍA SALIR MAL?).....	32
ANEXO — MAPA DE PREGUNTAS DE LA PAUTA	33

Introducción

Layla es una plataforma de escritura creativa colaborativa y worldbuilding dirigida a autores de ficción (novelistas independientes, grupos de roleo narrativo, equipos de guionistas). Permite que varias personas co-escriban un manuscrito en tiempo real, mantengan una wiki del universo de ficción, visualicen las relaciones entre entidades en un grafo narrativo y se comuniquen mediante una sesión de voz push-to-talk mientras trabajan. Este documento es el artefacto Especificación Funcional (AR02) del proyecto. Recoge el resultado de las fases de elicitación, análisis y especificación de requisitos, y sirve como contrato mínimo entre el equipo de desarrollo y el resto de stakeholders para las fases posteriores de diseño e implementación. El documento se organiza en siete secciones: propósito, alcance, definiciones, referencias, requisitos (contexto, clases de usuario, casos de uso, requisitos funcionales y no funcionales) y restricciones. Un anexo final mapea dónde se responde cada pregunta de la pauta del artefacto.

Propósito

El propósito de este documento es comunicar a todos los interesados (equipo de desarrollo, profesores/evaluadores, y futuros usuarios de la API) qué debe hacer el sistema Layla, para qué tipos de usuarios, bajo qué restricciones y con qué criterios de calidad.

Objetivos específicos:

- Establecer el contexto y el valor de negocio del producto.
- Enumerar las clases de usuario y los casos de uso que el sistema debe soportar.
- Detallar los requisitos funcionales y no funcionales trazables hasta los componentes ya implementados (endpoints REST, hubs de tiempo real, eventos asíncronos).
- Documentar las restricciones técnicas, académicas y regulatorias aceptadas por el equipo.

Este documento no cubre diseño detallado, plan de pruebas, ni despliegue en producción; esos artefactos se entregarán en etapas posteriores.

Alcance

La especificación cubre el sistema Layla end-to-end en su estado actual al cierre del ciclo de elicitación, incluye:

Dos servicios de backend:

- **server-core** (ASP.NET Core 10) — autenticación, usuarios, proyectos, roles, mensajería de tiempo real.
- **server-worldbuilding** (Node.js + Express 5) — manuscritos, wiki, grafo narrativo.

Tres clientes:

- Escritorio (WPF .NET 9).
- Web (Blazor .NET 9).

- Móvil (Android — Kotlin + Jetpack Compose).

Tres almacenes de datos:

- SQL Server (relacional).
- MongoDB (documentos).
- Neo4j (grafo).

Un broker de mensajería:

- RabbitMQ para eventos de dominio entre servicios.

Excluye:

- Diseño de UI de alta fidelidad, guías de estilo visual y kit de componentes (se documentan aparte).
- Planes de despliegue productivo, monitorización, DR/BCP y operaciones.
- Modelo comercial, acuerdos legales con creadores y gestión de pagos.
- Pruebas de aceptación, plan de QA, estrategia de release.

Definiciones, acrónimos y abreviaciones

Término	Significado
API	Application Programming Interface — interfaz de programación expuesta por un servidor.
JWT	JSON Web Token — token firmado usado como credencial de sesión.
RBAC	Role-Based Access Control — control de acceso basado en roles.
SignalR	Librería de ASP.NET para comunicación en tiempo real (WebSocket, SSE, long polling).
PTT	Push-to-Talk — modo de transmisión de voz en el que el usuario mantiene pulsado un botón para hablar.
RTF	Rich Text Format — formato de texto enriquecido usado para almacenar capítulos.
Outbox pattern	Patrón que garantiza que los eventos de dominio se publiquen tras confirmar la transacción de base de datos, evitando inconsistencias.
RabbitMQ	Broker AMQP usado como bus de mensajes asíncronos entre servicios.
CU	Caso de uso.
RF / RNF	Requisito funcional / Requisito no funcional.
Owner	Roles sobre un proyecto: propietario.
Editor	Roles sobre un proyecto: colaborador con permiso de edición.
Reader	Roles sobre un proyecto: lector.

Manuscrito	Agrupación ordenada de capítulos dentro de un proyecto.
Capítulo	Unidad de texto editable en formato RTF dentro de un manuscrito.
Wiki	Conjunto de entradas descriptivas sobre entidades del universo de ficción (personajes, lugares, objetos).
Grafo narrativo	Representación en Neo4j de entidades de la wiki y sus relaciones (nodos y aristas)
DAU	Daily Active Users — usuarios activos diarios

Referencias

Bibliografía

- Hunter, K. (2016). Irresistible APIs: Designing web APIs that developers will love. Manning.
- IEEE Std 830-1998. Recommended Practice for Software Requirements Specifications.

Documentación interna del proyecto

- README.md — arquitectura y referencia de API actual.
- REFACTORING_SUMMARY.md — cambios estructurales recientes (2026-03-23).
- docs/Layla (Documento de Proyecto).pdf — versión inicial del documento de proyecto (parcialmente vigente).

Documentación técnica externa

- Microsoft: ASP.NET Core 10 y SignalR.
- Node.js 22 y Express 5.
- MongoDB Document Model.
- Neo4j Cypher Query Language.
- RabbitMQ Topic Exchanges y AMQP 0-9-1.
- Jetpack Compose (Android).
- WPF con MaterialDesignInXAML / FluentWPF.

Contexto

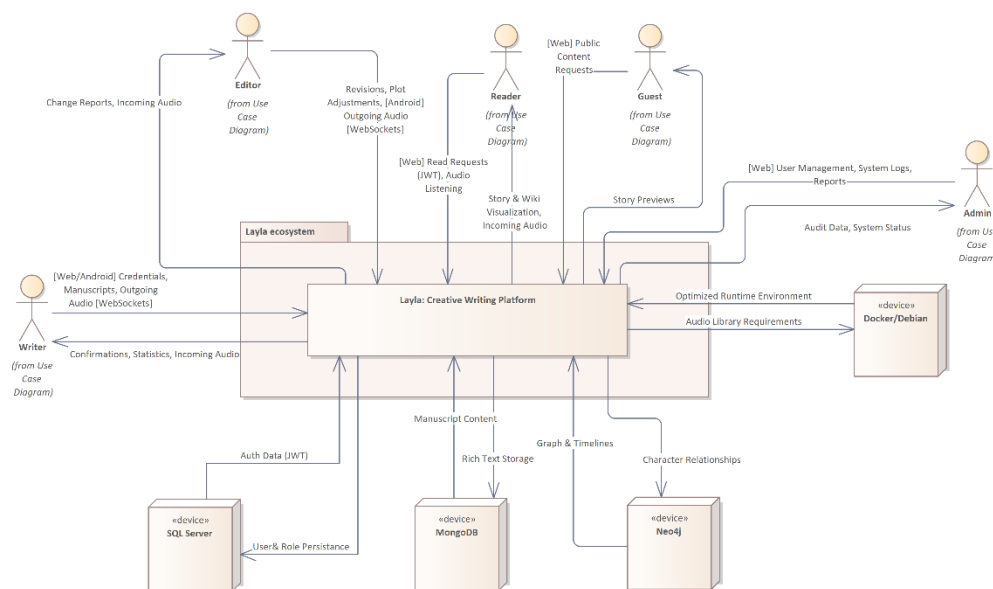
El ecosistema contemporáneo de la escritura creativa, el diseño de narrativas complejas y la construcción de mundos ficticios (*worldbuilding*) atraviesa una crisis estructural derivada de la profunda fragmentación de las herramientas digitales. Históricamente, la producción literaria y la gestión del canon narrativo han sido tratadas por la industria del software como procesos mutuamente excluyentes, lo que ha obligado a los autores y creadores a depender

de una amalgama de plataformas desconectadas para gestionar las múltiples dimensiones de su trabajo intelectual.

La redacción intensiva de manuscritos suele llevarse a cabo en procesadores de texto tradicionales o software especializado de escritorio de arquitectura monolítica (como Scrivener). Simultáneamente, la construcción de la enciclopedia del mundo se delega a plataformas basadas en la nube o complejas redes de notas (como World Anvil o Obsidian). El panorama competitivo actual ilustra esta dicotomía tecnológica, obligando a los usuarios a comprometer funciones críticas según la herramienta que seleccionen, limitando severamente la coautoría en tiempo real y la portabilidad de los datos.

Existe una ausencia crítica de una infraestructura de software integradora, resiliente y distribuida. La principal oportunidad reside en abordar las profundas limitaciones de interoperabilidad mediante la adopción de un modelo de persistencia políglota diseñado específicamente para el dominio narrativo. Al combinar bases de datos documentales para el texto enriquecido y bases de datos orientadas a grafos para el mapeo semántico, Layla se sitúa con una ventaja competitiva frente a soluciones de talla única.

composite structure Data Context Diagram



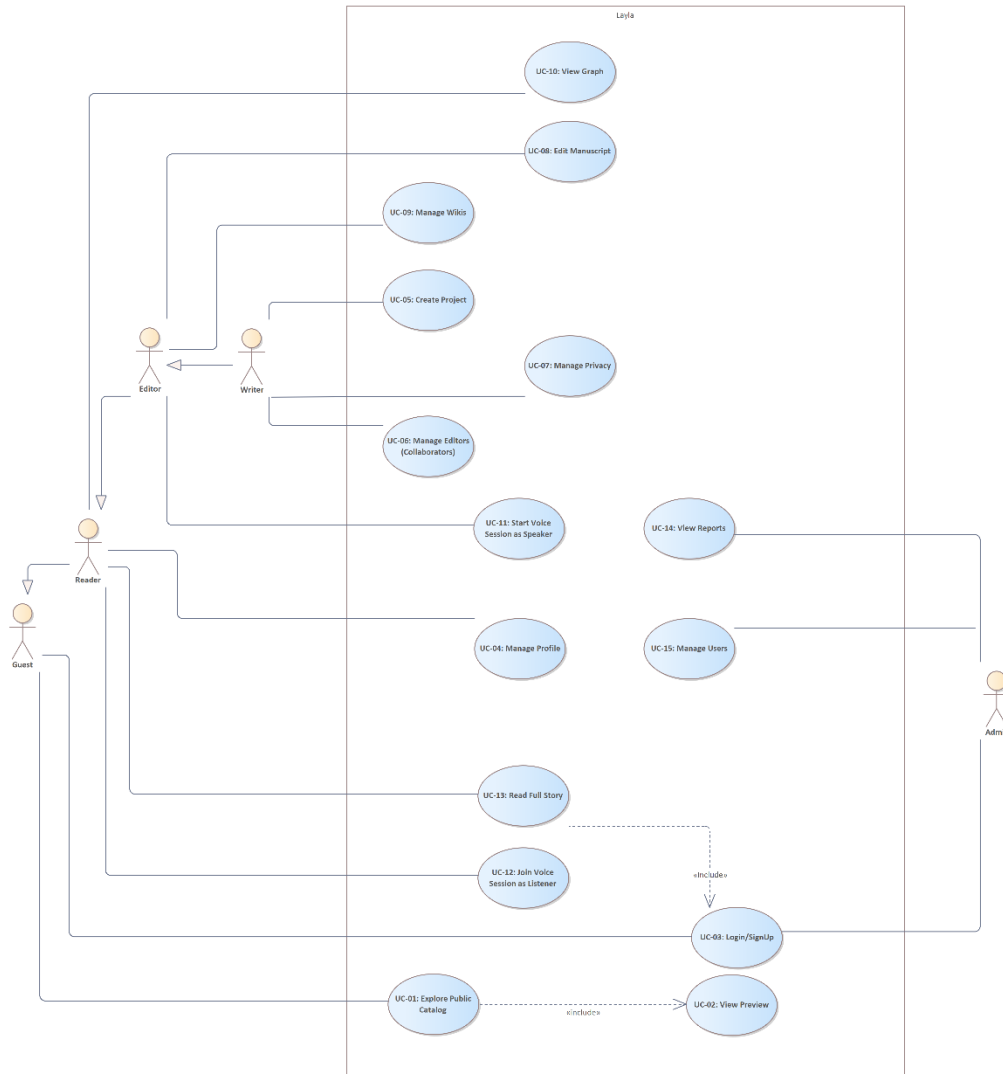
Clases de usuario

El sistema implementa un estricto Control de Acceso Basado en Roles (RBAC):

- **Administrador:** Pericia técnica alto, operador de la plataforma con permisos elevados, gestiona usuarios, baneo de cuentas y generación de reportes.
- **Escritor (Owner/Writer):** Pericia técnica media–Alta, familiarizado con procesadores de texto y herramientas colaborativas. Gestiona proyectos, redacta manuscritos y administra permisos. Tiene capacidad bidireccional en sesiones de voz.

- **Editor (Collaborator):** Usuario invitado para co-autoría. Puede modificar contenido y actualizar la wiki, con permisos de audio bidireccional.
- **Lector Registrado:** Consume contenido terminado y navega por grafos. En el módulo de voz, ingresa únicamente como Oyente.
- **Invitado (Guest):** Usuario no autenticado que explora exclusivamente el catálogo público (*Feed*) para incentivar su registro.

uc Use Case Diagram



Casos de uso

Definición

ID	Nombre del Caso de Uso	Actor Principal	Módulo
----	------------------------	-----------------	--------

CU-01	Explorar Catálogo Público (Feed)	Invitado	Acceso
CU-02	Ver Vista Previa (Sinopsis)	Invitado	Acceso
CU-03	Iniciar Sesión / Registrarse	Todos	Seguridad
CU-04	Gestionar Perfil	Usuario Registrado	Seguridad
CU-05	Crear Nuevo Proyecto	Escritor	Gestión
CU-06	Gestionar Colaboradores	Escritor	Gestión
CU-07	Configurar Privacidad	Escritor	Gestión
CU-08	Editar Manuscrito (Texto Rico)	Editor / Escritor	Escritura (MongoDB)
CU-09	Gestionar Wiki (Nodos)	Editor / Escritor	Worldbuilding (Neo4j)
CU-10	Visualizar Relaciones (Grafo)	Lector / Editor	Worldbuilding (Neo4j)
CU-11	Iniciar Sesión de Voz (Hablar)	Escritor / Editor	Voz (WebSockets)
CU-12	Unirse como Oyente	Lector	Voz (WebSockets)
CU-13	Leer Historia Completa	Lector	Lectura
CU-14	Ver Reportes del Sistema	Administrador	Admin
CU-15	Gestionar Usuarios (Ban/Roles)	Administrador	Admin

Clasificación

Módulo de Acceso y Seguridad

- **CU-03:** Iniciar Sesión / Registrarse
- **CU-04:** Gestionar Perfil
- **CU-15:** Gestionar Usuarios (Admin)

Módulo de Gestión y Escritura (Core)

- **CU-05:** Crear Nuevo Proyecto
- **CU-06:** Gestionar Colaboradores
- **CU-07:** Configurar Privacidad
- **CU-08:** Editar Manuscrito

Módulo de Worldbuilding (Grafos)

- **CU-09:** Gestionar Wiki
- **CU-10:** Visualizar Relaciones

Módulo de Voz y Colaboración

- **CU-11:** Iniciar Sesión de Voz (Hablar)
- **CU-12:** Unirse como Oyente

Módulo de Lectura Pública

- **CU-01:** Explorar Catálogo Público
- **CU-02:** Ver Vista Previa
- **CU-13:** Leer Historia Completa

Descripciones de casos de uso

Módulo de Acceso y Seguridad

Autenticación y Entrada

Nombre	CU-03 “Iniciar Sesión / Registrarse”
Responsable	Luis Donald Ortiz García
Fecha de actualización	13 de febrero de 2026
Descripción	El usuario (Invitado) ingresa sus credenciales para obtener un Token de Acceso (JWT) o crea una nueva cuenta en la plataforma para acceder a funciones restringidas.
Actor(es)	Invitado (Guest), Usuario Registrado.
Disparador	El usuario selecciona "Entrar" o "Registrarse" en la pantalla de bienvenida (Web o Desktop).
Precondiciones	PRE-1: El servicio de Backend Core (.NET) debe estar operativo. PRE-2: El usuario no debe tener una sesión activa.
Flujo Normal	1. El Sistema solicita correo electrónico y contraseña. 2. El Usuario ingresa sus credenciales. 3. El Sistema valida el formato del correo (FA-01). 4. El Usuario confirma el envío. 5. El Sistema verifica las credenciales contra SQL Server (hash de contraseña). 6. El Sistema genera un JSON Web Token (JWT) incluyendo el Rol del usuario (Escritor, Lector, etc.) y fecha de expiración. 7. El Sistema devuelve el token y redirige al Dashboard principal. 8. Termina CU.
Flujos Alternos	FA-01 Credenciales Inválidas: 1. El Sistema detecta que el correo no existe o la contraseña no coincide. 2. Muestra el mensaje: “Credenciales incorrectas”. 3. Incrementa el contador de intentos fallidos (Seguridad).

	4. Vuelve al paso 1 del FN. FA-02 Registro de Nuevo Usuario: 1. El Usuario selecciona "Crear Cuenta". 2. Ingresa Nombre, Correo y Password. 3. El Sistema valida que el correo no esté duplicado en SQL Server. 4. El Sistema crea el registro y loguea automáticamente al usuario (va al paso 6 del FN).
Excepciones	EX-1 Base de Datos No Disponible: 1. El Sistema muestra “Servicio de identidad no disponible temporalmente”. 2. Termina CU.
Postcondiciones	POS-1: El cliente (Web/WPF/Android) almacena el JWT en almacenamiento seguro (HttpOnly Cookie o SecureStorage). POS-2: El usuario tiene acceso a los recursos permitidos por su Rol.
Reglas de negocio	RN-01: Las contraseñas deben ser almacenadas utilizando un algoritmo de hash robusto (ej. BCrypt o Argon2). RN-02: El token JWT tiene una vigencia máxima de 24 horas.
Incluye	--
Extiende	--
Calidad de servicio	• Tiempo de respuesta autenticación < 500ms. • Confidencialidad (Tráfico cifrado vía HTTPS).

Gestión de Identidad Personal

Nombre	CU-04 “Gestionar Perfil”
Responsable	Luis Donaldo Ortiz García
Fecha de actualización	13 de febrero de 2026

Descripción	El usuario autenticado actualiza su información personal, cambia su contraseña o gestiona sus preferencias de notificación.
Actor(es)	Usuario Registrado (Cualquier rol).
Disparador	El usuario selecciona "Mi Perfil" en el menú de usuario.
Precondiciones	PRE-1: El usuario tiene una sesión activa (Token JWT válido).
Flujo Normal	1. El Sistema recupera los datos del usuario desde SQL Server usando el ID del token. 2. El Sistema muestra el formulario con Nombre, Biografía y Avatar. 3. El Usuario modifica los campos deseados. 4. El Usuario presiona "Guardar Cambios". 5. El Sistema valida la entrada (longitud de textos, formato de imagen). 6. El Sistema actualiza el registro en la base de datos. 7. El Sistema confirma la operación con un mensaje de éxito "Perfil Actualizado". 8. Termina CU.
Flujos Alternos	FA-01 Cambio de Contraseña: 1. El Usuario selecciona "Seguridad". 2. Ingresa contraseña actual y nueva contraseña (x2). 3. El Sistema valida que la contraseña actual sea correcta. 4. El Sistema actualiza el hash en la BD y revoca tokens anteriores (Opcional). 5. Termina CU.
Excepciones	EX-1 Sesión Expirada: 1. El Sistema detecta que el JWT ha caducado al intentar guardar. 2. Redirige al Login (CU-03).
Postcondiciones	POS-1: Los datos actualizados son visibles inmediatamente en todos los clientes sincronizados.
Reglas de negocio	RN-03: No se permite cambiar el correo electrónico principal sin una verificación de doble factor (email de confirmación).
Incluye	--
Extiende	--

Seguridad Administrativa

Nombre	CU-15 “Gestionar Usuarios (Ban/Roles)”
Responsable	Luis Donaldo Ortiz García
Fecha de actualización	13 de febrero de 2026
Descripción	El Administrador visualiza la lista de usuarios registrados y ejecuta acciones de seguridad como bloqueo de cuentas (Ban) o elevación de privilegios.
Actor(es)	Administrador (Admin).
Disparador	El Admin accede al "Panel de Usuarios" en el Cliente Web.
Precondiciones	PRE-1: El usuario debe tener el rol ROLE_ADMIN en su Token JWT. PRE-2: Acceso exclusivo desde Cliente Web (Dashboard).
Flujo Normal	1. El Sistema muestra una tabla paginada de usuarios (ID, Nombre, Correo, Estado, Rol). 2. El Admin busca un usuario por correo electrónico. 3. El Admin selecciona "Editar Permisos". 4. El Sistema muestra las opciones: "Bloquear Acceso" o "Cambiar Rol". 5. El Admin cambia el estado a "Bloqueado" (Banned) por violación de términos. 6. El Admin confirma la acción con su contraseña maestra. 7. El Sistema actualiza el estado en SQL Server. 8. El Sistema fuerza el cierre de sesión del usuario afectado (Blacklist de tokens). 9. Termina CU.
Flujos Alternos	FA-01 Promover a Moderador: 1. En el paso 4, el Admin cambia el rol de "Lector" a "Editor Global" o "Admin". 2. El Sistema actualiza los permisos.
Excepciones	EX-1 Auto-Ban: 1. El Sistema impide que un Administrador se bloquee o elimine a sí mismo.

	2. Muestra error: "Operación no permitida sobre su propio usuario".
Postcondiciones	POS-1: El usuario bloqueado recibe un error 403 Forbidden en su siguiente intento de acceso a la API.
Reglas de negocio	RN-04: Las acciones administrativas quedan registradas en un Log de Auditoría (Audit Trail).
Incluye	--
Extiende	--

Módulo de Gestión y Seguridad

Creación de Universos

Nombre	CU-05 “Crear Nuevo Proyecto”
Responsable	Luis Donaldo Ortiz García
Fecha de actualización	16 de febrero de 2026
Descripción	El Escritor inicializa un nuevo proyecto narrativo, estableciendo sus metadatos en SQL Server y su estructura de documentos en MongoDB.
Actor(es)	Escritor (Owner).
Disparador	El Escritor selecciona el botón flotante "+" en el Dashboard del Cliente WPF.
Precondiciones	PRE-1: El usuario debe estar autenticado. PRE-2: El servicio de Backend Core (.NET) debe estar disponible.
Flujo Normal	1. El Sistema solicita: Título, Sinopsis, Género Literario y Portada (Opcional). 2. El Escritor ingresa los datos. 3. El Sistema valida que el título no esté vacío (FA-01). 4. El Escritor confirma la creación. 5. El Sistema registra la relación User-Project en SQL Server con el rol OWNER. 6. El Sistema crea la colección de documentos vacía en MongoDB.

	7. El Sistema publica el evento ProjectCreated en RabbitMQ para inicializar el grafo en Neo4j. 8. El Sistema redirige al Escritor a la vista de "Escaleta" del nuevo proyecto. 9. Termina CU.
Flujos Alternos	FA-01 Título Vacío: 1. El Sistema resalta el campo en rojo y muestra "El título es obligatorio". 2. Vuelve al paso 2 del FN.
Excepciones	EX-1 Error de Transacción: 1. Si falla la creación en MongoDB después de insertar en SQL, el sistema hace rollback en SQL. 2. Muestra mensaje "Error de consistencia. Intente nuevamente".
Postcondiciones	POS-1: El proyecto aparece en la lista "Mis Proyectos" del usuario. POS-2: Se generan los permisos de administración (RBAC) para el creador.
Reglas de negocio	RN-01: Todo proyecto se crea por defecto como "Privado" (Borrador).
Incluye	--
Extiende	--
Calidad de servicio	· Integridad referencial garantizada.

Gestión de Equipo

Nombre	CU-06 “Gestionar Colaboradores”
Responsable	Luis Donaldo Ortiz García
Fecha de actualización	16 de febrero de 2026
Descripción	El Escritor invita a otros usuarios a participar en la obra, asignándoles roles específicos (Editor) para permitir la co-autoría o corrección.

Actor(es)	Escritor (Owner).
Disparador	El Escritor selecciona "Configuración del Equipo" en el menú del proyecto.
Precondiciones	PRE-1: El usuario debe ser el Propietario del proyecto (Role Owner).
Flujo Normal	1. El Sistema muestra la lista actual de colaboradores. 2. El Escritor ingresa el correo electrónico del usuario a invitar. 3. El Sistema busca al usuario en la base de datos de identidad (SQL Server). 4. El Sistema valida que el usuario exista (FA-01). 5. El Escritor selecciona el rol a asignar (ej. "Editor"). 6. El Sistema registra el permiso en la tabla de Roles del proyecto. 7. El Sistema envía una notificación al usuario invitado. 8. Termina CU.
Flujos Alternos	FA-01 Usuario no encontrado: 1. El Sistema muestra "No existe un usuario con ese correo". 2. Termina el flujo de invitación.
Excepciones	--
Postcondiciones	POS-1: El usuario invitado ahora puede ver y editar el proyecto en su propio Dashboard.
Reglas de negocio	RN-02: Un proyecto no puede tener más de 5 colaboradores en la versión gratuita (Opcional). RN-03: El Owner no puede ser eliminado del proyecto.
Incluye	--
Extiende	--

Control de Visibilidad

Nombre	CU-07 “Configurar Privacidad”
Responsable	Luis Donaldo Ortiz García

Fecha de actualización	16 de febrero de 2026
Descripción	El Escritor cambia la visibilidad del proyecto para publicarlo en el Feed o mantenerlo oculto.
Actor(es)	Escritor (Owner).
Disparador	El Escritor cambia el switch de "Estado del Proyecto".
Precondiciones	PRE-1: El proyecto debe tener al menos un capítulo y una sinopsis.
Flujo Normal	1. El Escritor selecciona cambiar estado de "Privado" a "Público". 2. El Sistema valida que el proyecto cumpla los requisitos mínimos (RN-04). 3. El Sistema solicita confirmación: "Al hacer público el proyecto, cualquier usuario podrá leerlo". 4. El Escritor confirma. 5. El Sistema actualiza la bandera IsPublic = true en SQL Server. 6. El Sistema indexa el proyecto en el motor de búsqueda del Feed Público. 7. Termina CU.
Flujos Alternos	--
Excepciones	--
Postcondiciones	POS-1: El proyecto es visible para el actor "Invitado" y "Lector".
Reglas de negocio	RN-04: No se puede publicar un proyecto vacío (sin capítulos).
Incluye	--
Extiende	--

Producción Literaria

Nombre	CU-08 “Editar Manuscrito (Texto Rico)”
Responsable	Luis Donaldo Ortiz García

Fecha de actualización	16 de febrero de 2026
Descripción	El usuario redacta o modifica el contenido narrativo de un capítulo utilizando el editor de texto enriquecido del Cliente WPF, con soporte para guardado offline.
Actor(es)	Escritor, Editor.
Disparador	El usuario hace doble clic en un capítulo desde la "Escaleta" en el Cliente WPF.
Precondiciones	PRE-1: El usuario tiene permisos de escritura (Writer/Editor) sobre el proyecto. PRE-2: El capítulo no está bloqueado por otro usuario (Concurrencia Optimista).
Flujo Normal	1. El Sistema carga el contenido del capítulo desde MongoDB (JSON/HTML). 2. El Sistema renderiza el texto en el control RichTextBox de WPF. 3. El Usuario realiza modificaciones (escribe, borra, aplica negritas). 4. El Sistema guarda automáticamente los cambios en caché local cada 30 segundos (FA-01). 5. El Usuario presiona "Guardar" o sale del capítulo. 6. El Cliente WPF envía el documento actualizado a la API (.NET). 7. La API actualiza el registro en MongoDB. 8. Termina CU.
Flujos Alternos	FA-01 Modo Offline (RF-18): 1. Si el Sistema detecta pérdida de red, activa el indicador "Guardando localmente". 2. Los cambios se escriben en un archivo local cifrado. 3. Al recuperar la red, el Sistema sincroniza automáticamente el archivo local con el servidor.
Excepciones	EX-1 Conflicto de Edición: 1. Si otro usuario modificó el mismo capítulo simultáneamente, el sistema detecta discrepancia de versiones. 2. Muestra ventana de "Resolución de Conflictos" (Guardar mi versión / Descartar).

Postcondiciones	POS-1: El contenido actualizado está disponible para los Lectores. POS-2: Se genera una nueva entrada en el historial de versiones (opcional).
Reglas de negocio	RN-05: El editor debe soportar formato RTF básico (Negrita, Cursiva, Subrayado) pero no scripts ni objetos incrustados inseguros.
Incluye	--
Extiende	--
Calidad de servicio	· Experiencia de escritura fluida (sin lag de red). · Persistencia garantizada ante caídas.

Módulo de Worldbuilding

Gestión de Entidades (Wiki)

Nombre	CU-09 “Gestionar Wiki (Nodos)”
Responsable	Luis Donaldo Ortiz García
Fecha de actualización	16 de febrero de 2026
Descripción	El usuario crea o edita una ficha de entidad (Personaje, Lugar, Objeto), almacenando su contenido descriptivo y creando su representación como nodo en el grafo.
Actor(es)	Escritor, Editor.
Disparador	El usuario selecciona "Agregar Entidad" o edita una existente en el panel de Wiki.
Precondiciones	PRE-1: El usuario tiene permisos de escritura (Writer/Editor). PRE-2: El servicio de Node.js (Worldbuilding) está operativo.
Flujo Normal	1. El Sistema solicita el Tipo de Entidad (Personaje, Ubicación, Evento, Objeto). 2. El Usuario ingresa los atributos: Nombre, Alias, Descripción y Etiquetas.

	3. El Usuario guarda la ficha. 4. El Sistema valida que el nombre no esté duplicado en el proyecto (FA-01). 5. El Backend Node.js crea/actualiza el nodo en Neo4j (para relaciones). 6. El Backend Node.js guarda el cuerpo de texto extenso en MongoDB. 7. El Sistema confirma la operación. 8. Termina CU.
Flujos Alternos	FA-01 Nombre Duplicado: 1. El Sistema detecta un nodo con el mismo nombre. 2. Muestra alerta: "Ya existe 'Arturo'. ¿Desea editarlo o crear un homónimo?". 3. El usuario elige la acción.
Excepciones	EX-1 Fallo Parcial (Riesgo R-01): 1. Si Neo4j está caído, el sistema guarda en MongoDB y encola el evento en RabbitMQ para crear el nodo después. 2. Notifica al usuario: "Guardado. El grafo se actualizará en breve".
Postcondiciones	POS-1: La entidad es buscable en la Wiki y aparece visualmente en el grafo.
Reglas de negocio	RN-01: Todo nodo debe pertenecer obligatoriamente a un Proyecto (Tenant Isolation).
Incluye	--
Extiende	--

Visualización del Grafo

Nombre	CU-10 “Visualizar Relaciones (Grafo)”
Responsable	Luis Donald Ortiz García
Fecha de actualización	16 de febrero de 2026
Descripción	El usuario visualiza la red de conexiones entre los elementos de la historia (quién conoce a quién, dónde ocurrió qué evento) mediante una interfaz gráfica interactiva.

Actor(es)	Escritor, Editor, Lector.
Disparador	El usuario accede a la pestaña "Mapa de Relaciones" o "Grafo".
Precondiciones	PRE-1: El proyecto debe tener al menos dos entidades y una relación creada.
Flujo Normal	1. El Usuario solicita ver el grafo. 2. El Cliente (Web/WPF) solicita los datos al servicio de Node.js. 3. El Sistema recupera los nodos y aristas desde Neo4j usando el protocolo Bolt. 4. El Sistema renderiza el grafo visualmente (Nodos como círculos, Relaciones como flechas). 5. El Usuario hace clic en un nodo. 6. El Sistema despliega un panel lateral con el resumen de la ficha (traído de MongoDB). 7. Termina CU.
Flujos Alternos	FA-01 Filtrado: 1. El Usuario selecciona "Ver solo Personajes". 2. El Sistema filtra la consulta Cypher y actualiza la vista.
Excepciones	--
Postcondiciones	POS-1: El usuario obtiene una comprensión visual de la estructura de la historia.
Reglas de negocio	RN-02 (Permisos): Los Lectores y Editores solo pueden ver; no pueden arrastrar ni modificar nodos si el grafo está en modo "Bloqueado" por el Escritor.
Incluye	--
Extiende	--

Módulo de Voz y Colaboración

Transmisión de Audio (Habla)

Nombre	CU-11 "Iniciar Sesión de Voz (Hablar)"
Responsable	Luis Donaldo Ortiz García

Fecha de actualización	16 de febrero de 2026
Descripción	El usuario con rol de colaborador participa activamente en una sesión de voz utilizando el mecanismo Push-to-Talk para transmitir sus ideas.
Actor(es)	Escritor, Editor (Rol Emisor).
Disparador	El usuario presiona el icono de micrófono en la App Android.
Precondiciones	PRE-1: El usuario debe tener permiso de Writer o Editor en el proyecto. PRE-2: Permiso de uso de micrófono concedido en el sistema operativo (Android).
Flujo Normal	1. El Sistema conecta al usuario al Hub de WebSockets del proyecto. 2. El Sistema habilita el botón de "Push-to-Talk" (PTT). 3. El Usuario mantiene presionado el botón PTT. 4. El Cliente Android captura el audio, lo codifica y lo envía en <i>chunks</i> binarios al servidor. 5. El Sistema muestra indicador visual "Transmitiendo" a todos los usuarios. 6. El Usuario suelta el botón. 7. El Sistema corta la transmisión. 8. Termina CU.
Flujos Alternos	FA-01 Canal Ocupado: 1. Si otro usuario está hablando, el botón PTT se deshabilita. 2. El Sistema muestra "Canal ocupado por [Nombre Usuario]".
Excepciones	EX-1 Desconexión (RNF-08): 1. Si se pierde la conexión WiFi, la app intenta reconectar 3 veces. 2. Si falla, muestra "Desconectado del chat de voz" y deshabilita el botón PTT.
Postcondiciones	POS-1: El mensaje de audio fue distribuido a todos los oyentes conectados en tiempo real.

Reglas de negocio	RN-03 (Half-Duplex): Solo un usuario puede transmitir a la vez para evitar eco y saturación.
Incluye	--
Extiende	--

Recepción de Audio (Oyente)

Nombre	CU-12 “Unirse como Oyente”
Responsable	Luis Donald Ortiz García
Fecha de actualización	16 de febrero de 2026
Descripción	Un usuario (Lector o Editor pasivo) ingresa a la sala de voz para escuchar la sesión de trabajo sin intervenir activamente.
Actor(es)	Lector, Editor, Escritor (Rol Receptor).
Disparador	El usuario selecciona "Escuchar Sesión" en el Cliente Web o Android.
Precondiciones	PRE-1: El proyecto tiene una sesión de voz activa.
Flujo Normal	1. El Sistema valida el token JWT del usuario. 2. El Sistema establece la conexión WebSocket en modo "Solo Escucha". 3. El Servidor agrega al usuario a la lista de distribución del Hub. 4. Cuando un "Emisor" habla (CU-11), el Sistema recibe los paquetes de audio. 5. El Cliente (Navegador/Android) reproduce el audio en secuencia. 6. El Sistema muestra visualmente quién está hablando. 7. Termina CU.
Flujos Alternos	FA-01 Silenciar (Mute): 1. El Usuario Oyente decide silenciar la salida de audio localmente. 2. El Sistema deja de reproducir sonido, pero mantiene la conexión al socket.
Excepciones	--

Postcondiciones	POS-1: El usuario aparece en la lista de "Audiencia" del proyecto.
Reglas de negocio	RN-04: Un Lector nunca puede activar el micrófono (el sistema no solicita permiso de Micrófono para este rol).
Incluye	--
Extiende	--

Módulo de Lectura Pública

Descubrimiento de Contenido

Nombre	CU-01 “Explorar Catálogo Público”
Responsable	Luis Donaldo Ortiz García
Fecha de actualización	16 de febrero de 2026
Descripción	El usuario navega por el "Feed" o catálogo de descubrimiento, visualizando las portadas y sinopsis de las obras que han sido marcadas como públicas por sus autores.
Actor(es)	Invitado (Guest), Lector.
Disparador	El usuario accede a la página de inicio (/home) o selecciona "Explorar" en el menú.
Precondiciones	PRE-1: El servicio de Backend Core (.NET) y la base de datos SQL Server deben estar operativos.
Flujo Normal	1. El Sistema consulta en SQL Server los proyectos con la bandera IsPublic = true. 2. El Sistema recupera los metadatos (Título, Autor, Género, URL de Portada). 3. El Sistema renderiza una cuadrícula (Grid) de tarjetas de proyectos. 4. El Usuario hace scroll para ver más obras. 5. El Sistema carga más elementos bajo demanda (Paginación o Infinite Scroll). 6. Termina CU.
Flujos Alternos	FA-01 Filtrado por Género:

	1. El Usuario selecciona una categoría (ej. "Ciencia Ficción"). 2. El Sistema filtra la consulta SQL. 3. El Sistema actualiza la cuadrícula mostrando solo obras coincidentes.
Excepciones	EX-1 Catálogo Vacío: 1. Si no hay obras públicas, el sistema muestra un mensaje amigable: "Sé el primero en publicar una historia".
Postcondiciones	--
Reglas de negocio	RN-01: El catálogo solo muestra obras que tengan al menos un capítulo publicado y una sinopsis definida.
Incluye	--
Extiende	--

Atracción

Nombre	CU-02 “Ver Vista Previa”
Responsable	Luis Donaldo Ortiz García
Fecha de actualización	16 de febrero de 2026
Descripción	El usuario accede a la ficha técnica de una obra para leer su sinopsis y evaluar si desea comenzar la lectura completa.
Actor(es)	Invitado (Guest).
Disparador	El usuario hace clic en una portada desde el Catálogo (CU-01).
Precondiciones	PRE-1: El proyecto seleccionado debe ser público.
Flujo Normal	1. El Sistema carga la página de detalle del proyecto. 2. El Sistema muestra: Título, Autor, Sinopsis completa, Género y Etiquetas. 3. El Sistema muestra un botón de llamada a la acción (CTA): "Leer Ahora" o "Registrarse para Leer". 4. El Usuario lee la información. 5. Termina CU.

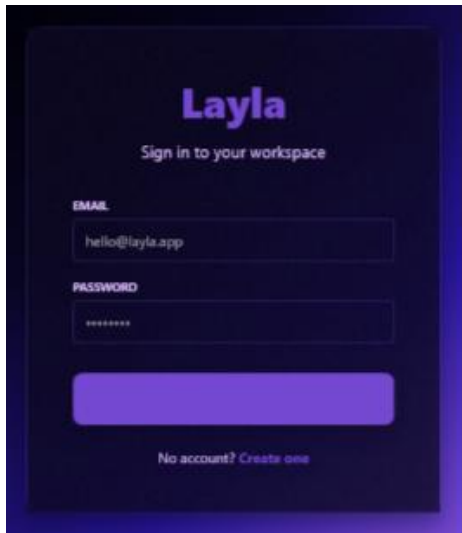
Flujos Alternos	FA-01 Usuario ya Registrado: 1. Si el sistema detecta una sesión activa (JWT), el botón cambia a "Continuar Lectura" y lleva al CU-13.
Excepciones	--
Postcondiciones	--
Reglas de negocio	RN-02 (Acceso Limitado): Un usuario Invitado NO puede ver el contenido de los capítulos ni el grafo de relaciones, solo la metainformación.
Incluye	CU-03 "Iniciar Sesión / Registrarse" (Opcional, si el usuario decide entrar).
Extiende	--

Consumo de Contenido

Nombre	CU-13 “Leer Historia Completa”
Responsable	Luis Donaldo Ortiz García
Fecha de actualización	16 de febrero de 2026
Descripción	El usuario autenticado accede a la interfaz de lectura inmersiva, visualizando el manuscrito completo y consultando la Wiki lateralmente.
Actor(es)	Lector (Registered Reader).
Disparador	El usuario selecciona "Leer" en la vista previa de un proyecto.
Precondiciones	PRE-1: El usuario debe estar autenticado (Token JWT válido). PRE-2: El proyecto debe ser público o el usuario debe ser colaborador.
Flujo Normal	1. El Sistema carga la estructura de capítulos (Índice) desde MongoDB. 2. El Sistema renderiza el texto del primer capítulo en modo "Solo Lectura" (sin barras de herramientas de edición). 3. El Usuario avanza en la lectura.

	4. El Usuario hace clic en un nombre de personaje resaltado (Link inteligente). 5. El Sistema despliega un panel lateral con la ficha del personaje traída desde la Wiki (Neo4j / Mongo). 6. Termina CU.
Flujos Alternos	FA-01 Navegación por Índice: 1. El Usuario abre el menú de capítulos. 2. Selecciona "Capítulo 5". 3. El Sistema carga asíncronamente el contenido de ese documento.
Excepciones	EX-1 Contenido Bloqueado: 1. Si el autor cambia la obra a "Privada" mientras un lector lee, el sistema permite terminar el capítulo actual, pero bloquea el siguiente.
Postcondiciones	POS-1: El sistema registra el progreso de lectura (último capítulo visto) en el perfil del usuario.
Reglas de negocio	RN-03 (Integridad): El Lector tiene estrictamente prohibida la modificación del texto; la interfaz no debe cargar los scripts de edición de texto enriquecido.
Incluye	--
Extiende	CU-10 "Visualizar Relaciones" (El lector puede ver el mapa si el autor lo permite).

Prototipo IU

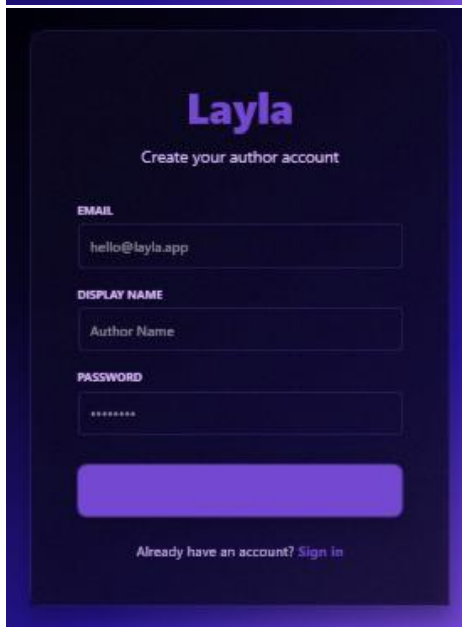


Layla
Sign in to your workspace

EMAIL
hello@layla.app

PASSWORD

[No account? Create one](#)



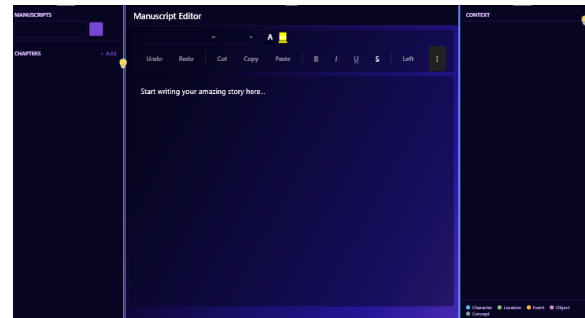
Layla
Create your author account

EMAIL
hello@layla.app

DISPLAY NAME
Author Name

PASSWORD

[Already have an account? Sign in](#)



MANUSCRIPTS

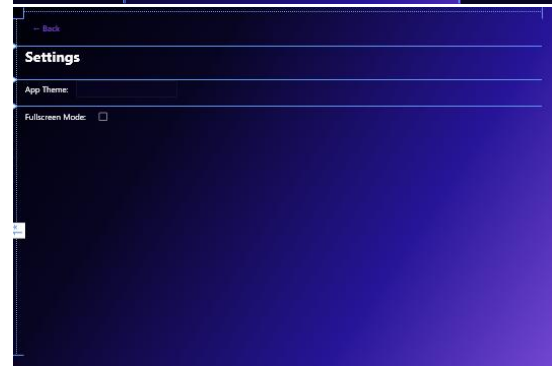
CHAPTERS

Manuscript Editor

Undo Redo Cut Copy Paste Bold Italic Underline Link

Start writing your amazing story here...

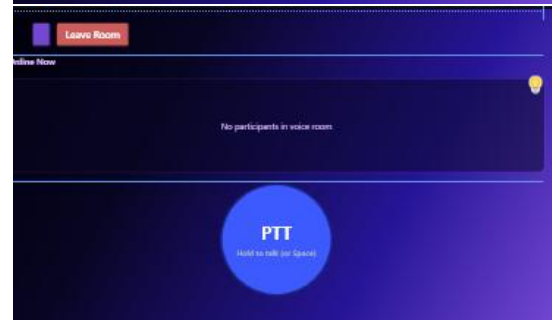
CONTEXT



Settings

App Theme:

Fullscreen Mode: ☐

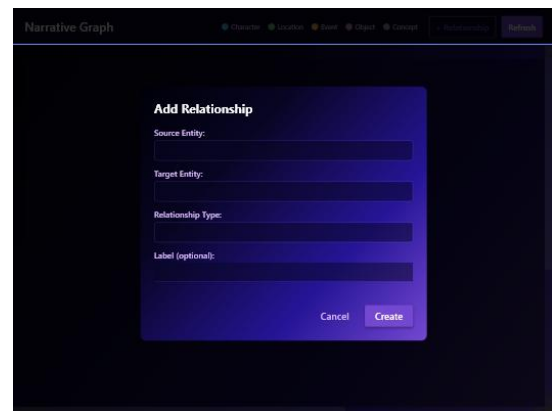


Leave Room

Online Now

No participants in voice room

PTT
Hold to talk (tap Space)



Narrative Graph

Overview Location Event Object Concept Relationships Subgraph

Add Relationship

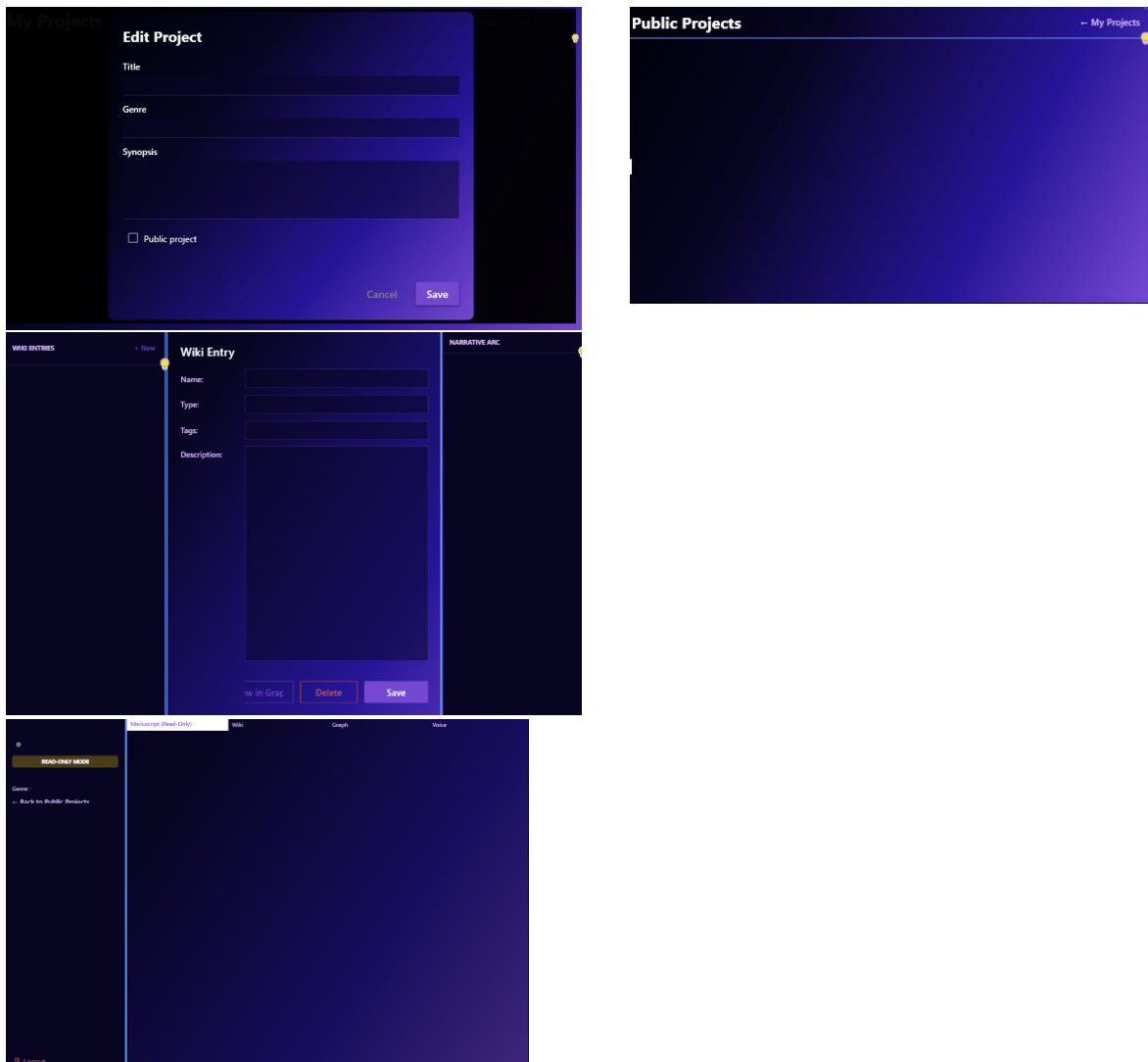
Source Entity:

Target Entity:

Relationship Type:

Label (optional):

Cancel Create



Requisitos funcionales

Módulo de Gestión de Usuarios y Seguridad (Backend .NET + SQL Server)

- **RF-A01: Autenticación y Autorización.** El sistema debe permitir el registro e inicio de sesión de usuarios, generando un Token (JWT) que identifique su rol (Escritor, Editor, Lector, Admin) y permisos de sesión.
- **RF-A02: Gestión de Perfiles.** El sistema debe permitir a los usuarios actualizar su información básica y gestionar sus credenciales de acceso almacenadas en SQL Server.
- **RF-A03: Control de Acceso Basado en Roles (RBAC).** El sistema debe restringir el acceso a los módulos de edición y voz según la relación del usuario con el proyecto (ej. un Lector no puede editar; un Editor no puede borrar la obra).
- **RF-A04: Registro en el sistema.** El sistema permitirá a un usuario registrarse proporcionando email, contraseña y nombre para mostrar (DisplayName).
- **RF-A05: Eliminación de cuenta.** El sistema permitirá al usuario eliminar su propia cuenta.

Módulo de Escritura y Proyectos (Backend Node.js + MongoDB)

- **RF-B01: Gestión de Proyectos Narrativos.** El sistema permitirá crear un proyecto con título, sinopsis, género literario, imagen de portada y visibilidad pública/privada. El creador queda como Owner .
- **RF-B02: Modificación de metadatos de proyecto.** El sistema permitirá al Owner actualizar los metadatos del proyecto.
- **RF-B03: Eliminación de proyecto.** El sistema permitirá al Owner eliminar el proyecto propagando el borrado a MongoDB y Neo4j.
- **RF-B04: Publicación de eventos persistente.** El sistema publicará el evento project.created en RabbitMQ tras el commit transaccional de SQL Server (Outbox Pattern).
- **RF-B05: Obtención de proyectos por tipo de rol.** El sistema listará los proyectos del usuario, los proyectos públicos y, para administradores, todos los proyectos

Módulo de Worldbuilding y Relaciones (Backend Node.js + Neo4j)

- **RF-C01: Gestión de la Wiki del Mundo.** El sistema debe permitir la creación de fichas de personajes, lugares y objetos, almacenando sus atributos en nodos de una base de datos orientada a grafos (Neo4j).

- **RF-C02: Vinculación de Entidades.** El sistema debe permitir establecer y visualizar relaciones semánticas entre nodos (ej. "Personaje A es *hermano de* Personaje B") para generar el grafo de la historia.

Módulo de Colaboración y Voz (Android + WebSockets)

- **RF-D01: Comunicación por Voz (Push-to-Talk).** El sistema implementará un modelo de comunicación por voz tipo *Walkie-Talkie* o *Broadcast* para optimizar el ancho de banda y garantizar la entrega de paquetes UDP/WebSockets. Esto mitiga problemas de eco y latencia en redes inestables.
- **RF-D02: Roles de Voz.** El sistema distinguirá entre roles de "Emisor" (Escritor/Editor activo) y "Oyente" (Lectores/Invitados), gestionando los canales de audio mediante ASP.NET Core SignalR.
- **RF-D03: Notificaciones de Presencia.** El sistema debe indicar visualmente qué usuarios están conectados y editando o hablando dentro de un proyecto activo.
- **RF-D04: Unión a proyecto.** El sistema permitirá a un usuario unirse a un proyecto público como Reader.

Módulo de Lectura y Acceso Público (Web Razor Pages)

- **RF-E01: Visualización de Contenido Público.** El portal Web debe permitir a usuarios invitados (Guest) visualizar la sinopsis y fragmentos de obras marcadas como públicas sin necesidad de autenticación.
- **RF-E02: Lectura Inmersiva.** El sistema debe presentar al Lector registrado una interfaz limpia de lectura del manuscrito completo, con acceso lateral a la Wiki del mundo.

Módulo de Administración y Reportes (Web Razor Pages)

- **RF-F01: Monitoreo del Sistema.** El sistema debe proveer al Administrador un tablero con el estado de los contenedores Docker y la salud de los servicios.
- **RF-F02: Generación de Reportes.** El sistema debe generar gráficas estadísticas sobre el uso de la plataforma (número de proyectos, usuarios activos, volumen de datos) para la toma de decisiones.
- **RF-F03: Catálogo de Obras Públicas (Feed).** El sistema debe presentar una vista de "Descubrimiento" en el portal web (Razor Pages) que liste las portadas y sinopsis de los proyectos marcados como públicos. Esta lista debe permitir el filtrado por género (ej. Fantasía, Sci-Fi) y ordenamiento por "Más Recientes".
- **RF-F04: Sincronización Híbrida.** El cliente WPF debe ser capaz de operar con caché local, permitiendo al escritor continuar redactando en caso de micro-cortes de red y sincronizando los cambios con el Backend (.NET) automáticamente al restablecerse la conexión.

- **Criterio de Aceptación:** Para resolver conflictos de edición simultánea o desfasada (offline), el sistema implementará la política de "**Última Escritura Gana**" (**Last Write Wins - LWW**). Se utilizará la marca de tiempo del servidor (Server-Side Timestamp) al momento de recibir la petición para determinar qué versión del manuscrito prevalece, descartando versiones anteriores que lleguen con retraso.
- **RF-F05: Procesamiento Asíncrono de Datos.** Las operaciones pesadas de escritura en grafos y documentos extensos deben gestionarse mediante colas de mensajes, asegurando que la interfaz de usuario no se congele esperando la confirmación de la base de datos secundaria.

Requisitos no funcionales

Operabilidad y Despliegue

Rendimiento

- **RNF-01: Latencia.** La latencia p95 de endpoints CRUD deberá ser < 300 ms bajo carga nominal
- **RNF-02: Escritura con perdida.** La edición de capítulos no mantendrá locks prolongados — se adopta Last-Write Wins como trade-off explícito

Disponibilidad:

- **RNF-03: Chequeos de estado.** Los servicios expondrán healthchecks (/health , /api/health) consumibles por Docker y orquestadores.
- **RNF-04: Tasa de disponibilidad mensual.** Objetivo de disponibilidad del 99 % mensual en entorno productivo.

Seguridad:

- **RNF-05: Autenticación.** Autenticación por JWT firmado, con TokenVersion que permita invalidación inmediata.
- **RNF-06: Validación de JWT.** Validación fail-fast en arranque: JWT_SECRET y JWT_SECRET_REFRESH deben tener al menos 32 caracteres.
- **RNF-07: Rate limiting.** Rate limiting por IP en endpoints de autenticación.
- **RNF-08: Accesos permitidos.** CORS restringido a orígenes autorizados.
- **RNF-09: RBAC.** RBAC en todos los endpoints de proyecto (Owner , Editor , Reader).

Mantenibilidad:

- **RNF-10: Clean code.** server-core sigue arquitectura limpia: Layla.Api → Layla.Core → Layla.Infrastructure.
- **RNF-11: Imports estables.** server-worldbuilding usa alias de ruta @/ para imports estables.
- **RNF-12: Categorización de errores.** Todos los errores se modelarán con la enum tipada ErrorCode (27 códigos), mapeada a HTTP status por ApiControllerBase.RespondWithError(ErrorCode?).

Interoperabilidad

- **RNF-13: Publicación de documentación de API.** Ambos backends expondrán Swagger/OpenAPI (/swagger y /api-docs).
- **RNF-14: Unificación de un formato de comunicación.** Todos los intercambios serán JSON UTF-8.

Escalabilidad

- **RNF-15: Desacoplamiento asíncrono de servicios.** Los servicios estarán desacoplados por RabbitMQ; se adopta el patrón Outbox para garantizar consistencia eventual.
- **RNF-16: Buen flujo de apagado de servidores.** server-worldbuilding implementará graceful shutdown sobre SIGTERM / SIGINT , cerrando ordenadamente HTTP, RabbitMQ y Neo4j.

Usabilidad:

- **RNF-17: Aplicación de bibliotecas para crear interfaces de usuario más amigables.** El cliente de escritorio usará Material Design / FluentWPF para una experiencia visual coherente con Windows 11.
- **RNF-18: Percepción de velocidad beneficiada.** El cliente de escritorio funcionará con caché local y sincronización diferida cuando la red no esté disponible.

Portabilidad:

- **RNF-19: Generalización en contenedores.** Todos los servicios se distribuirán como imágenes Docker basadas en Debian Slim, orquestadas por docker-compose .

Restricciones

Restricciones académicas (impuestas por el curso):

- Se requieren **3 clientes diferenciados**: escritorio, web y móvil.
- Se requieren **3 bases de datos** con modelos diferentes: relacional, documental y grafo.

- Se requieren **≥ 2 backends**, uno en .NET y otro en un stack distinto, comunicados por un broker. La documentación de entrega se produce sobre la plantilla oficial del equipo.

Restricciones técnicas:

- Stack fijado: .NET 10, .NET 9, Node.js 22 + Express 5, SQL Server, MongoDB 7, Neo4j 5, RabbitMQ 3.
- El JWT caduca a las 24 h y requiere re-login (sin refresh token automático en MVP).
- Puertos reservados: 5288 (HTTPS core), 5287 (HTTP core), 3000 (worldbuilding).
- Los identificadores de entidad son GUID en server-core y ObjectId / uuid en server-worldbuilding .

Restricciones de negocio

- Plazos académicos estrictos (calendario Eminus) — sin margen para re-scoping.
- Equipo pequeño y sin presupuesto para servicios cloud de pago → despliegue local y self-hosted.
- El producto debe poder demostrarse offline (evaluaciones presenciales).

Restricciones regulatorias y de propiedad intelectual

- Los manuscritos son propiedad intelectual del autor; el sistema nunca los redistribuye fuera del proyecto sin consentimiento.
- Se ofrecerá derecho al olvido (GDPR-like): al eliminar una cuenta se borran o anonimizan los datos personales.
- Los proyectos privados quedan estrictamente cerrados al Owner y sus colaboradores.

Riesgos (¿qué podría salir mal?)

Riesgo	Mitigación
Pérdida de eventos entre server-core y server-worldbuilding por caída de RabbitMQ.	Outbox Pattern (publicación tras <i>commit</i>); reintentos idempotentes en el consumidor.
Conflicto de edición simultánea en un capítulo.	Estrategia <i>Last-Write-Wins</i> aceptada; en fases posteriores se evaluará CRDT u <i>operational transforms</i> .
Grafos grandes en Neo4j degradan la visualización.	Paginación de aristas; <i>layout</i> perezoso en el cliente.
Latencia de voz en redes móviles.	Paquetes de audio pequeños + <i>backoff</i> ; degradar a chat si PTT no es viable.
Filtrado de contenido inapropiado en salas de voz.	Moderación reactiva (<i>mute</i> / baneo por Owner o Administrador); política de uso.
Token JWT filtrado.	TokenVersion permite invalidación inmediata; caducidad de 24 h acota la ventana.

Mal comportamiento de un Owner (expulsar a colaboradores masivamente).	Registro de auditoría; CU-14 (reportes) como canal de revisión.
--	---

Anexo — Mapa de preguntas de la pauta

Pregunta de la guía	Sección(es) que la responden
¿Cuál es el problema que el proyecto está resolviendo?	§6.1 — <i>Problema que resuelve Layla.</i>
¿Cuál es el valor del negocio?	§6.1 — <i>Valor de negocio.</i>
¿Cuáles son las métricas y los casos de uso?	§6.3 — Casos de uso y §6.6 — <i>Métricas para medir éxito.</i>
¿Qué recursos se necesitan disponibles?	§6.6 — <i>Recursos necesarios</i> y §6.7 — <i>Restricciones técnicas.</i>
¿Qué podría salir mal?	§6.7 — <i>Riesgos.</i>